

Treinamento ETJ 2024-2025

Este documento tem como objetivo especificar o conteúdo abordado no treinamento da Eyetech Júnior, previsto para o período de novembro de 2024 a fevereiro de 2025. O treinamento está dividido em dois níveis: **Básico** e **Avançado**, abrangendo tópicos essenciais tanto para o desenvolvimento de back-end quanto de front-end, além de boas práticas, configurações para otimizar o fluxo de trabalho e práticas de engenharia de software.

Nível Básico

- Importância de seguir padrões e boas práticas de desenvolvimento.
- Métodos para pesquisar e aprender padrões e boas práticas das tecnologias.
- Criação e configuração de ambientes de desenvolvimento.
- Instalação e gerenciamento de bibliotecas de terceiros.
- Controle de versionamento utilizando Git/GitHub.
- Estratégias para pesquisar e aprender novas tecnologias de forma eficiente.
- Identificação de ferramentas que auxiliam no processo de desenvolvimento.
- Compreensão do fluxo básico de funcionamento de uma aplicação web.
- Manipulação inicial de dados com técnicas recomendadas de lógica de programação.
- Introdução às APIs: definição, métodos e funcionamento.
- Manipulação básica do DOM com JavaScript para criação de páginas dinâmicas.

Nível Avançado

Aqui você encontrará três áreas principais de especialização: **Back-End**, **Front-End** e **Engenharia de Software**. Normalmente, ao escolher um caminho, você optará por se aprofundar em uma dessas áreas, de acordo com seus interesses e o foco de sua carreira. No entanto, vale destacar que sempre há a possibilidade de estudar duas, ou até mesmo as três áreas, se assim desejar. O aprendizado contínuo e a busca por novos desafios são incentivados, e você pode, ao longo do tempo, ampliar seus conhecimentos para se tornar um profissional ainda mais completo e versátil.

Back-End

PHP

- Compreensão básica da linguagem e sua sintaxe.
- Programação Orientada a Objetos.
- Utilização do operador `->` para acessar propriedades e métodos.
- Encadeamento de Métodos.

- Propriedades Dinâmicas.
- Injeção de Dependência.
- Composer: o Gerenciador de Dependências do PHP
 - Instalação e configuração.
 - Gerenciamento de dependências.
 - Autoloading.
- Configuração do PHP (php.ini)
 - Habilitação de Extensões.
 - Configurações Comuns.

Laravel

- Compreensão do padrão de design MVC (Model-View-Controller).
- Instalação e configuração do Laravel.
- Estrutura de pastas e arquivos no Laravel.
 - Pastas principais: `app`, `routes`, `resources`, `database`, `public`, etc.
 - Arquivo `env` e configuração de variáveis de ambiente.
- Rotas e Controllers:
 - Definição de rotas (GET, POST, PUT, DELETE).
 - Parâmetros de rota e validação.
 - Controladores (Controllers) e Actions.
 - Resource Controllers e métodos RESTful.
- Models e Eloquent ORM:
 - Definição de Models e interação com o banco de dados.
 - Relacionamentos (One-to-One, One-to-Many, Many-to-Many).
 - Eloquent Queries: `where`, `join`, `orderBy`, etc.
- Factories e Seeders:
 - Como criar `factories` para popular o banco de dados com dados de teste.
 - Uso de `seeders` para carregar dados automaticamente.
- Migrations:
 - Criação e modificação de tabelas no banco de dados.
 - Comandos `php artisan migrate` e afins.
 - Compreensão das colunas, índices e constraints.
- Resources:
 - Definição de **API Resources** para formatar e estruturar respostas JSON.
 - Transformadores e como aplicar os recursos de exibição.
- Validadores:
 - Criação de regras de validação para entrada de dados.
 - Validação no controller com `validate()` ou `Validator::` e personalização de mensagens.
 - Validação de dados em formulários e APIs.
- Autenticação e Autorização com Laravel Breeze para API:

- Instalação e configuração do Laravel Breeze para API com Sanctum.
- Configuração de rotas e controllers de autenticação de API.
- Geração de tokens de autenticação com Sanctum

Segurança

- Gerenciamento de cookies.
- CSRF (Cross-Site Request Forgery).
- CORS (Cross-Origin Resource Sharing).
- SQL Injection.
- XSS (Cross-Site Scripting).
- Configuração de Headers de Segurança.
- Rate Limiting.
- Cuidados com a exposição de dados sensíveis.
- Controle de sessões.
- Segurança em APIs.

Front-End

HTML

- Estrutura semântica e boas práticas.
- Compreensão das tags principais: `<div>`, `<section>`, `<header>`, `<footer>`, `<main>`, etc.
- Acessibilidade (WAI-ARIA, atributos como `alt`, `aria-label`).

CSS

- Box Model, Flexbox e Grid.
- Diferenças entre classes, IDs e seletores compostos.
- Conhecimento de pseudo-classes (`:hover`, `:nth-child`) e pseudo-elementos (`::before`, `::after`).
- Uso de variáveis CSS (`--main-color`) e suas aplicações.

JavaScript (ES6+)

- Manipulação de variáveis com `let` e `const`.
- Uso de arrow functions e template literals.
- Manipulação do DOM e eventos básicos (`addEventListener`, `querySelector`).

Gerenciamento de Pacotes

- Compreensão geral do funcionamento do **NPM** (Node Package Manager).
- Instalação de pacotes com `npm install`.
- Diferença entre **dependências** (`dependencies`) e **dependências de desenvolvimento** (`devDependencies`).

- Exploração de alternativas como **Yarn** ou **PNPM** (Performant Node Package Manager) para melhorar a performance.

Vite

- Diferenças entre **Vite** e **Webpack**.
- Como o **Vite** otimiza o carregamento de módulos e acelera o desenvolvimento.
- Configuração básica no arquivo `vite.config.js`.
- Uso de aliases para simplificar imports.

CommonJS vs ES Modules

- Diferenças entre `require` (CommonJS) e `import/export` (ES Modules).
- Uso de módulos ECMAScript em projetos modernos.

TypeScript

- O que é o **TypeScript** e suas diferenças em relação ao **JavaScript**.
- Arquivo `tsconfig.json`: como configurar a compilação do TypeScript.
- Tipos básicos: `string`, `number`, `boolean`, `any`, `void`, `null`, `undefined`.
- Inferência de tipos.
- Tipos personalizados.
- Funções tipadas.
- **Interfaces** e **Tipos**.
- **Generics**.

CSS Moderno

- Uso do **TailwindCSS** para estilização utilitária.
- Personalização do tema no arquivo `tailwind.config.js`.
- Responsividade utilizando prefixes como `sm:`, `md:`, `lg:`.

SASS

- O que é o **SASS** e as diferenças em relação ao CSS comum.
- Uso de variáveis, mixins e funções para reutilização de código.
- **Nesting** (aninhamento) para criar hierarquias claras.
- Organização modular com `@import` ou `@use`.

React (TypeScript + Vite)

- Conceitos Básicos
 - **TSX**: mistura de HTML com TypeScript.
 - Criação de **componentes funcionais** e reutilizáveis.
 - Passagem de dados via **props** e uso de **children**.
- Hooks
 - Uso do `useState` para gerenciamento de estados simples.
 - Uso do `useEffect` para lidar com efeitos colaterais.
 - Como criar **hooks personalizados**.
- Ciclo de Vida e Atualizações
 - Compreensão de quando os componentes são montados, atualizados e desmontados.

- Gerenciamento de Estado
 - Estado Global
 - Conceito e gerenciamento do estado global.
 - Utilização de **Zustand** ou **Redux**.
 - Estado HTTP
 - Conceito e gerenciamento do estado HTTP.
 - Utilização do **TanStack Query** (anteriormente React Query).
- Renderização
 - Uso de **React.memo** para evitar renderizações desnecessárias.
 - Hooks como `useCallback` e `useMemo` para memoizar funções e valores.
- Load States
 - Uso de **Skeleton loaders** para fornecer feedback visual enquanto os dados são carregados.
 - Implementação de estados de erro e sucesso claros, com mensagens para o usuário.
- Rotas
 - Compreensão de como construir as rotas no lado do cliente.
 - Uso do **React Router**.

Engenharia de Software

Engenharia de Requisitos

- Levantamento de requisitos: técnicas e boas práticas.
- Análise de requisitos funcionais e não funcionais.
- Definição clara dos requisitos do sistema para garantir a entrega de valor ao cliente.
- Ferramentas para documentação de requisitos

Documentação de Software

- Importância da documentação técnica e do código.
- Como criar documentação clara e objetiva para sistemas e APIs.
- Práticas de versionamento de documentação junto com o código-fonte.
- Ferramentas e técnicas para gerar documentação automática

Diagramas UML (Unified Modeling Language)

- Introdução ao UML e sua importância na modelagem de sistemas.
- **Diagramas de Casos de Uso:** como mapear as interações do usuário com o sistema.
- **Diagramas de Classe:** representação das classes e seus relacionamentos.
- **Diagramas de Sequência:** representação das interações entre objetos ao longo do tempo.
- **Diagramas de Atividades:** fluxo de controle do sistema e processos de negócios.
- **Diagramas de Componentes:** estrutura modular do sistema.

- **Diagramas de Implantação:** arquitetura física e distribuição dos componentes.

Boas Práticas de Engenharia de Software

- Ciclo de vida de desenvolvimento de software (SDLC).
- Testes unitários, integração e de sistema.
- Controle de versões: Git Flow, branches e pull requests.
- Revisões de código e práticas de peer review.
- Refatoração de código e melhoria contínua.
- Planejamento de sprint e metodologias ágeis (ex.: Scrum, Kanban).

OBS: Foi definido como padrão a utilização do Figma, por isso é de extrema importância estudar e aprender a como utiliza-lo.

A equipe diretora da Eyetech Júnior estará sempre à disposição para apoiar e orientar durante todo o processo de treinamento. Nosso compromisso é oferecer a melhor experiência de aprendizado e proporcionar o suporte necessário para esclarecer qualquer dúvida sobre os conteúdos abordados. Se em algum momento você precisar de auxílio, seja para resolver questões técnicas ou para entender melhor algum conceito, não hesite em nos procurar. Estamos aqui para ajudar no seu desenvolvimento e garantir que sua jornada de aprendizado seja bem-sucedida.